UNIVERSITY OF MSILA
COMPUTER SCIENCE DEPARTMENT

L2-INF/INF-D
Object-oriented programming I
Problem set 1: Procedural Programming and Its Limitations

04/10/2026
two weeks

Exercise 1: Limits of procedural programming

Consider the following C program, which manages a small array of students (grades are marks out of 20).

```c
#include <stdio.h>
#include <string.h>
#define NB_STUDENTS 4
#define NAME_LEN 10
typedef struct {
  char name[NAME_LEN];
  int age;
  float grade;
} Student;

void init_student(Student *s, char *name, int age, float grade) {
  strncpy(s->name, name, NAME_LEN - 1);
  s->name[NAME_LEN - 1] = '\0';
  s->age = age;
  s->grade = grade;
}

void print_student(Student *s) {
  printf("%-30s | age: %3d | grade: %5.2f\n", s->name, s->age, s->grade);
}

void print_all(Student tab[], int n) {
  for (int i = 0; i < n; i++)
    print_student(&tab[i]);
}

float average_grade(Student tab[], int n) {
  float sum = 0.0f;
  for (int i = 0; i < n; i++)
    sum += tab[i].grade;
  return (n > 0) ? sum / n : 0.0f;
}

int main(void) {
  Student students[NB_STUDENTS];
  init_student(&students[0], "Ali", 18, 24);
  init_student(&students[1], "Omar", 19, 7.75);
  init_student(&students[2], "Amira", 18, 13);
  init_student(&students[3], "Ahlem", -20, 8.5);
  print_all(students, NB_STUDENTS);
  printf("\nAverage grade: %.2f\n", average_grade(students, NB_STUDENTS));
  return 0;
}
```

Part 1: Observing the Program

1. Compile and run the program.

Solution:

```
Ali   | age:  18 | grade: 24.00
Omar  | age:  19 | grade:  7.75
Amira | age:  18 | grade: 13.00
Ahlem | age: -20 | grade:  8.50

Average grade: 13.31
```

2. Examine the output. What is wrong with Ahlem's age and Ali's grade?

Solution: Ahlem's age (-20) is negative, and Ali's grade (24) is above 20: neither value makes sense for a student.

3. What type of error do these invalid values represent?

Solution: Logic (semantic) errors: each value is legal for its C type but meaningless for the problem. The compiler cannot detect them and the program does not crash.

Part 2: Implementing Data Validation

4. Modify `init_student()` so that it checks the validity of the student's data. If the data is invalid, the program must exit with a message indicating the error (you may need `#include <stdlib.h>`):
   - the age must be between 1 and 100;
   - the grade must be between 0 and 20.

Solution: Add the following code at the top of `init_student()`:

```
if (age < 1 || age > 100) {
  printf("wrong age %d for student: %s\n", age, name);
  exit(-1);
}
if (grade < 0 || grade > 20) {
  printf("wrong grade %.2f for student: %s\n", grade,
      name);
  exit(-1);
}
```

5. Run the program again. Correct the invalid values in `main()` one by one until the program runs to completion.

Solution: The program stops at the first invalid value, Ali's grade: `wrong grade 24.00 for student: Ali`. Once it is corrected, it stops at Ahlem: `wrong age -20 for student: Ahlem`. The program only runs to the end when all the data are valid.

Part 3: The Limits of Validation

In `main()`, just before the call to `print_all()`, add:

```
students[0].grade = 57.5f;
students[1].age = -3;
strcpy(students[2].name, "Boubakri Taki eddin");
```

6. Run the program again and verify the results. What do you notice?

Solution:

```
Ali                 | age:  18 | grade: 57.50
Omar                | age:  -3 | grade:  7.75
Boubakri Taki eddin | age: 1684349033 | grade:
      0.00
Ahlem               | age:  20 | grade:  8.50

Average grade: 18.44
```

Ali's grade (57.50) and Omar's age (-3) are invalid, yet nothing complains. Changing Amira's name also changed her age and grade to unexpected values (the exact values depend on the platform), and the average changed.

7. Does anything in the `Student` structure prevent a programmer from writing the following?

```
students[0].grade = 100;
students[0].age = -50;
```

Solution: No. The fields are directly accessible, and `int` and `float` accept any value of their type: the structure only groups data, it protects nothing.

8. We updated `init_student()` to check the data. Why can't it help in this case?

Solution: It is just a function that the programmer must remember to call: the validation is separate from the data, and a direct assignment bypasses it. Moreover, it only checks values at initialization.

9. The `strcpy()` call only changes Amira's name. Why did her age and the class average change as well?

Solution: `strcpy()` does not know the size of the destination: it copies 20 bytes (19 characters plus the final `'\0'`) into the 10 bytes of `name`. The excess overwrites the next members of the same structure (padding, `age`, `grade`). Here it stops exactly at the end of the structure; a longer name would reach the next record.

10. What type of error is this?

Solution: A buffer overflow: a memory error (undefined behavior) detected neither by the compiler nor at run time. Unlike the two previous errors, it silently corrupts unrelated data.

Exercise 2: Moving to C++

Exercise 1 showed the limits of procedural programming. Before addressing them, we first migrate the program to C++ and use a few features that make C code cleaner and safer. Copy your validated program from Exercise 1 (without the lines added in Part 3) into a new `exo2.cpp` and compile it

Part 1: Constants and Types

1. Replace the two `#define` constants with typed constants, e.g. `constexpr int NB_STUDENTS = 4;`. Give two advantages of a typed constant over a macro.

Solution:

```
constexpr int NB_STUDENTS = 4;
constexpr int NAME_LEN    = 10;
```

A typed constant is checked by the compiler (type), respects scopes, is visible to the debugger, and involves no blind textual substitution.

2. In C++, a `struct` name is already a type name. Rewrite the declaration of `Student` without `typedef`.

Solution:

```
struct Student {
  char  name[NAME_LEN];
  int   age;
  float grade;
};
```

Part 2: References and `const`

3. Make `init_student()` take a *reference* (`Student &s`) instead of a pointer. Update the body (`s.age` instead of `s->age`) and the calls (no more `&`). Why don't the calls need `&` anymore? Name two differences between a reference and a pointer.

Solution:

```
void init_student(Student &s, const char *name,
                  int age, float grade);
init_student(students[0], "Ali", 21, 14);
```

A reference is an alias of the argument: the compiler passes its address implicitly. Unlike a pointer, a reference must be initialized, cannot be null, cannot be reassigned, and uses `.` instead of `->` (no explicit dereference).

4. Make `print_student()` take a `const Student &s`. Add `s.age = 0;` to its body and compile. What happens, and when is the error detected?

Solution: Compile-time error: `assignment of member 'Student::age' in read-only object`. `const` lets the compiler enforce read-only access before the program ever runs.

5. Which functions of the program should take `const` parameters, and which should not? Justify your answer.

Solution: `print_student()`, `print_all()` and `average_grade()` only read the data: `const`. `init_student()` modifies its parameter: no `const`. Rule: use `const` whenever a function does not modify its parameter.

Part 3: Overloading and Default Arguments

6. Rename both `print_student()` and `print_all()` to `print()`. Why does the program still compile? How does the compiler choose which function to call?

Solution: Several functions may share a name if their parameter lists differ (`print(const Student &)` and `print(const Student[], int)`). The compiler picks the one matching the number and types of the arguments, at compile time. This is impossible in C, where a name designates one function.

7. Give `init_student()` default values for the age and the grade (`int age = 18, float grade = 0`), then call `init_student(students[2], "Amira", 17);` and `init_student(students[3], "Ahlem");`. Why must default arguments come last in the parameter list?

Solution: Amira gets the default grade (0) and Ahlem the default age (18) and grade. Arguments are matched left to right: if a defaulted parameter came before a required one, the compiler could not tell which arguments were omitted. Once a parameter has a default, all the following ones must have one too.

Exercise 3: From `struct` to `class`

Exercise 2 made the code cleaner, but nothing prevents the errors of Exercise 1, Part 3. Start again from your `exo2.cpp`.

1. Move the `print()` function that prints one student inside `Student` as a member function without parameter, declared `void print() const`, and call it with `tab[i].print()`. Which parameter has disappeared, and how does the function reach the data? What happens if you remove the `const`?

Solution:

```
struct Student {
  char name[NAME_LEN]; int age; float grade;
  void print() const {
    printf("%-30s | age: %3d | grade: %5.2f\n",
           name, age, grade);
  }
};
```

The student is no longer a parameter: the function is called *on* an object and receives its address implicitly (`this`), so `age` means `this->age`. Without `const`, the array version of `print()` no longer compiles (`passing 'const Student' as 'this' argument discards qualifiers`): a `const` member function promises not to modify the object.

2. Replace `struct` by `class`, keep `print()` public and move the validation into public functions `setAge()`, `setGrade()` and `setName()`; add `getGrade() const` for `average_grade()`, and let `init_student()` call the setters. Then try `students[1].age = -3;` in `main()`. What happens, and why can the validation no longer be bypassed?

Solution:

```
class Student {
  char name[NAME_LEN]; int age; float grade;
public:
  void print() const { /* as before */ }
  float getGrade() const { return grade; }
  void setAge(int age) {
    if (age < 1 || age > 100) { /* message */
      exit(-1);
    }
    this->age = age;
  }
  void setGrade(float grade) { /* same, 0..20 */ }
  void setName(const char *name) {
    strncpy(this->name, name, NAME_LEN - 1);
    this->name[NAME_LEN - 1] = '\0';
  }
};
```

Compile-time error: `'int Student::age' is private within this context`. The members are private, so the only way to change them is through the functions that validate them: the checks are attached to the data instead of being a separate function the programmer must remember to call. The same holds for the name: the `strcpy()` of Exercise 1 no longer compiles.

3. Which problems of Exercise 1 are now prevented? What can still go wrong?

Solution: Invalid values (compile time for direct writes, run time through the setters) and the buffer overflow (the bounded copy is the only way to set the name) are prevented. A programmer can still forget to call `init_student()`, which leaves an uninitialized `Student`. Making initialization automatic is the role of *constructors*.